IIA MVP Documentation

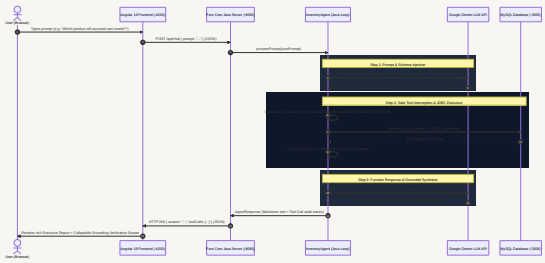
Here is the complete architectural walkthrough, data transformation lifecycle, LLM interaction mechanics, and codebase tour for the entire application.

---

1. High-Level Architecture & End-to-End Workflow

This application uses the **Model Context Protocol (MCP) / Dynamic Function Calling** pattern.

Unlike traditional chatbots that use rigid predefined templates or hardcoded SQL templates, this architecture gives the LLM complete autonomy to inspect your relational schema, dynamically generate exact multi-table SQL queries, inspect the raw results, and synthesize an executive business assessment.



---

2. Data Flow & Type Transformations (JSON ⇄ Java ⇄ SQL)

Understanding the data journey across language boundaries is the core of this system:

```
[User Browser Input]
       |
       ▼ (String)
[Angular Frontend (TypeScript)]
       |
       ▼ (HTTP POST JSON Payload)
[Pure Java HttpServer: HttpExchange]
       |
       ▼ (org.json.JSONObject in Java)
```

```
[Gemini REST API Call (HTTP JSON)]
       |
       ▼ (functionCall argument: SQL String)
[ToolExecutor / DatabaseManager]
       |
       ▼ (JDBC Statement executeQuery(sql))
[MySQL Storage Engine]
       |
       ▼ (JDBC java.sql.ResultSet)
[ResultSetMetaData → org.json.JSONArray of JSONObjects]
       |
       ▼ (Gemini functionResponse JSON part)
[Gemini LLM Context Window]
       |
       ▼ (Natural Language Markdown String)
[Angular UI & Grounding Inspector]
```

Transformation Step-by-Step:

1. **User Prompt (TypeScript String → HTTP JSON)**:
   In [`app.ts`](file:///Users/sherlock/Downloads/inventory-agent-app/frontend/src/app/app.ts#L158-L163), the user input string is packaged into a JSON request:
   ```
   { "prompt": "which product is likely to succeed in the market next month" }
   ```
2. **Core Java Ingestion (HTTP JSON → Java Objects)**:
   [`HttpUtil.readRequestBody(exchange)`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/util/HttpUtil.java) reads the raw input stream into a string and converts it into an `org.json.JSONObject`.
3. **Schema & Tool Assembly (Java → Gemini Wire Format)**:
   [`InventoryAgent.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/agent/InventoryAgent.java#L40-L84) builds the Gemini API payload:
   - **`systemInstruction`**: Injects the full DDL definition of all 6 MySQL tables (`categories`, `suppliers`, `warehouses`, `products`, `inventory_stock`, `stock_movements`).
   - **`tools`**: Registers `execute_inventory_query` using standard OpenAPI-style schemas declared in [`ToolDefinition.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/gemini/ToolDefinition.java).
   - **`contents`**: The conversation history so far.

4. **Gemini's Autonomous Decision (Gemini JSON → Java `functionCall`)**:
   Gemini decides that answering this question requires inspecting stock movements and gross margins. Instead of emitting plain text, it returns:

```
{
  "functionCall": {
    "name": "execute_inventory_query",
    "args": {
      "query": "SELECT p.product_name, (p.unit_price - p.cost_
price) AS unit_margin, SUM(ist.quantity_on_hand) AS total_stoc
k FROM products p JOIN inventory_stock ist ON p.product_id = i
st.product_id GROUP BY p.product_id, p.product_name, p.unit_pr
ice, p.cost_price ORDER BY unit_margin DESC;",
      "explanation": "Analyze inventory stock and profit margi
ns to determine high-potential products."
    },
    "id": "call_12345"
  },
  "thoughtSignature": "EmAKXgFpFH0..."
}
```

5. **SQL Execution Guardrail (Java String → MySQL Native Binary Protocol)**:
   [`DatabaseManager.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/db/DatabaseManager.java#L89-L138) intercepts the SQL string:
   - **Security Check**: Queries *must* begin with `SELECT`, `SHOW`, `WITH`, or `DESCRIBE`. Any statement containing destructive commands (`DROP`, `DELETE`, `UPDATE`, `INSERT`, `ALTER`, `TRUNCATE`) throws an immediate `SecurityException`.
   - **JDBC Execution**: Sends the validated query to MySQL on port `3306` with a strict 10-second query timeout.
6. **Relational Data Extraction (JDBC `ResultSet` → Java `JSONArray`)**:
   `DatabaseManager` loops through the `ResultSetMetaData`:

```
ResultSetMetaData metaData = rs.getMetaData();
int columnCount = metaData.getColumnCount();
while (rs.next() && rowCount < 100) {
    JSONObject row = new JSONObject();
```

```
            for (int i = 1; i <= columnCount; i++) {
                row.put(metaData.getColumnLabel(i), rs.getObject(i));
            }
            results.put(row);
        }
```

This converts SQL rows into an array of JSON objects (the **extracted sub-table**).

7. **Function Response Feedback (Sub-table JSON → Gemini Context Window)**:
   In [`InventoryAgent.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/agent/InventoryAgent.java#L128-L150):
   - Gemini requires that the exact candidate content (including its `thoughtSignature` and `id`) be added to the conversation history.
   - Then, a turn with `role: "user"` containing `functionResponse` is appended, wrapping the JSON sub-table.
8. **Executive Synthesis (Gemini Context → Structured Markdown)**:
   Gemini reads the sub-table it just requested, detects patterns (margins, stock velocity, warehouse locations), and writes an executive assessment report.
9. **Frontend Rendering (Server Response → UI Display)**:
   The Angular frontend receives the Markdown analysis and the tool execution traces. It formats the markdown into clean typography, while placing the raw SQL queries and data grids inside an expandable **"Verify Grounding & SQL Audit"** drawer.

---

3. What the Tool & Agent Are Doing

- **The Agent (`InventoryAgent`)**:
  - Acts as the **orchestrator / workflow loop manager**.
  - Maintains the state of the conversation.
  - Ensures that tool executions don't loop indefinitely (capped at 5 turns).
  - Directs Gemini to produce an executive-ready business report rather than raw database dumps.
- **The Tool (`InventoryTool` / `ToolExecutor`)**:
  - Acts as the **safe database bridge**.

- Publishes its schema to Gemini (`execute_inventory_query` with a `query` parameter and an `explanation` parameter).
- Measures query runtime in milliseconds (`executionTimeMs`).
- Enforces query timeouts, row limits (max 100 rows per query), and read-only constraints.

---

4. Comprehensive Codebase Tour

Backend Components (`backend/`)

1. [`App.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/App.java)

- **Pure Core Java Server Entrypoint**:
  Strictly utilizes `com.sun.net.httpserver.HttpServer` from the standard Java library (`java.base`), avoiding Spring Boot or heavy frameworks.
- **Executor Configuration**: Configures an unbounded/pooled executor (`Executors.newVirtualThreadPerTaskExecutor()` or cached pool) to handle concurrent HTTP requests without thread starvation.
- **Route Mapping**:
  - `POST /api/chat` → [`ChatHandler`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/handler/ChatHandler.java)
  - `GET/POST /api/db/status` & `/api/db/config` → [`DatabaseStatusHandler`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/handler/DatabaseStatusHandler.java)
  - `GET/POST /api/config` → [`ConfigHandler`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/handler/ConfigHandler.java)
  - `GET /api/inventory/sample` & `/api/inventory/schema` → [`SampleDataHandler`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/handler/SampleDataHandler.java)
  - `GET /api/health` → Health check endpoint

2. [`DatabaseManager.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/db/DatabaseManager.java)

- **Connection Pooling**: Utilizes HikariCP (`HikariDataSource`) with a 5-connection pool for low latency.
- `executeReadOnlyQuery(String sql)`:
  - Enforces read-only safety checks.
  - Enforces a 10-second JDBC statement timeout.
  - Enforces a 100-row cap per tool call.
- `getSystemStatus()`:
  - Physically queries `SHOW TABLES`, `COUNT(*)` of products, and `SUM(quantity_on_hand)`.
  - Returns true connection metrics (e.g. `13 products, 2263 stock units across 6 tables`) or the exact, unwrapped JDBC exception on failure.

3. [`GeminiClient.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/gemini/GeminiClient.java)

- **Standard HTTP/2 Client**: Built using `java.net.http.HttpClient`.
- **Model Fallback Chain**:
  Automatically manages model selection:
  1. Primary: `gemini-3.5-flash-lite` (active, unexhausted free quota).
  2. Secondary: `gemini-flash-lite-latest`.
  3. Tertiary: `gemini-3-flash-preview`.
- **Resilience**: If any model returns `429 RESOURCE_EXHAUSTED` (such as the 20 RPD cap on `gemini-3.6-flash`), `503 UNAVAILABLE`, or `404 NOT_FOUND`, it automatically cascades to the next available model without failing the user request.

4. [`InventoryAgent.java`](file:///Users/sherlock/Downloads/inventory-agent-app/backend/src/main/java/com/inventory/agent/InventoryAgent.java)

- **The Agent Core Loop**:
  Manages up to 5 execution turns.
  - Injects system schema instructions.

- Intercepts `functionCall`.
- Preserves `thoughtSignature` to adhere to Gemini 2.5/3.x protocol requirements.
- If the model performs multiple query turns without outputting an analysis, it runs an executive synthesis prompt turn so the user always receives a complete business report.

Frontend Components (`frontend/`)

1. [`app.ts`](file:///Users/sherlock/Downloads/inventory-agent-app/frontend/src/app/app.ts)

- **Angular 19 Standalone Component**:
  Uses Angular Signals (`messages = signal<ChatMessage[]>([])` , `dbStatus = signal<DbStatus>(...)` , `dbFeedback` , `isThinking`).
- `sendMessage(prompt)` :
  Dispatches the user's natural language prompt to `POST /api/chat` , manages loading indicators, and receives `{ answer, toolCalls }` .
- `openDbModal()` & `saveDbConfig()` :
  Communicates with `POST /api/db/config` without closing the modal, displaying live database metrics (`tableCount` , `totalProducts` , `totalStockUnits`) or the exact MySQL driver error.
- `formatMarkdown(text)` :
  A lightweight Markdown parser that converts headers (`#` , `##` , `###`), lists (,), bold text (`*`), and tables into clean HTML tags without heavy external libraries.

2. [`app.html`](file:///Users/sherlock/Downloads/inventory-agent-app/frontend/src/app/app.html)

- **Top Navigation Bar**: Live status pills displaying the Core Java server status (`:8080`), MySQL database connection with item counts (`13 items`), and the active Gemini model.
- **Executive Analysis Stream**: Displays Gemini's synthesized business analysis as primary cards.
- **Grounding Verification Bar**: A collapsible inspection drawer below every agent response. Users who want to verify the underlying data can expand this drawer to

view:

  - The exact generated SQL query.
  - Execution time in milliseconds.
  - The raw extracted sub-table in a data grid.
- **Real-time MySQL Modal**: Displays authentic live verification cards, table counts, stock numbers, and JDBC status.

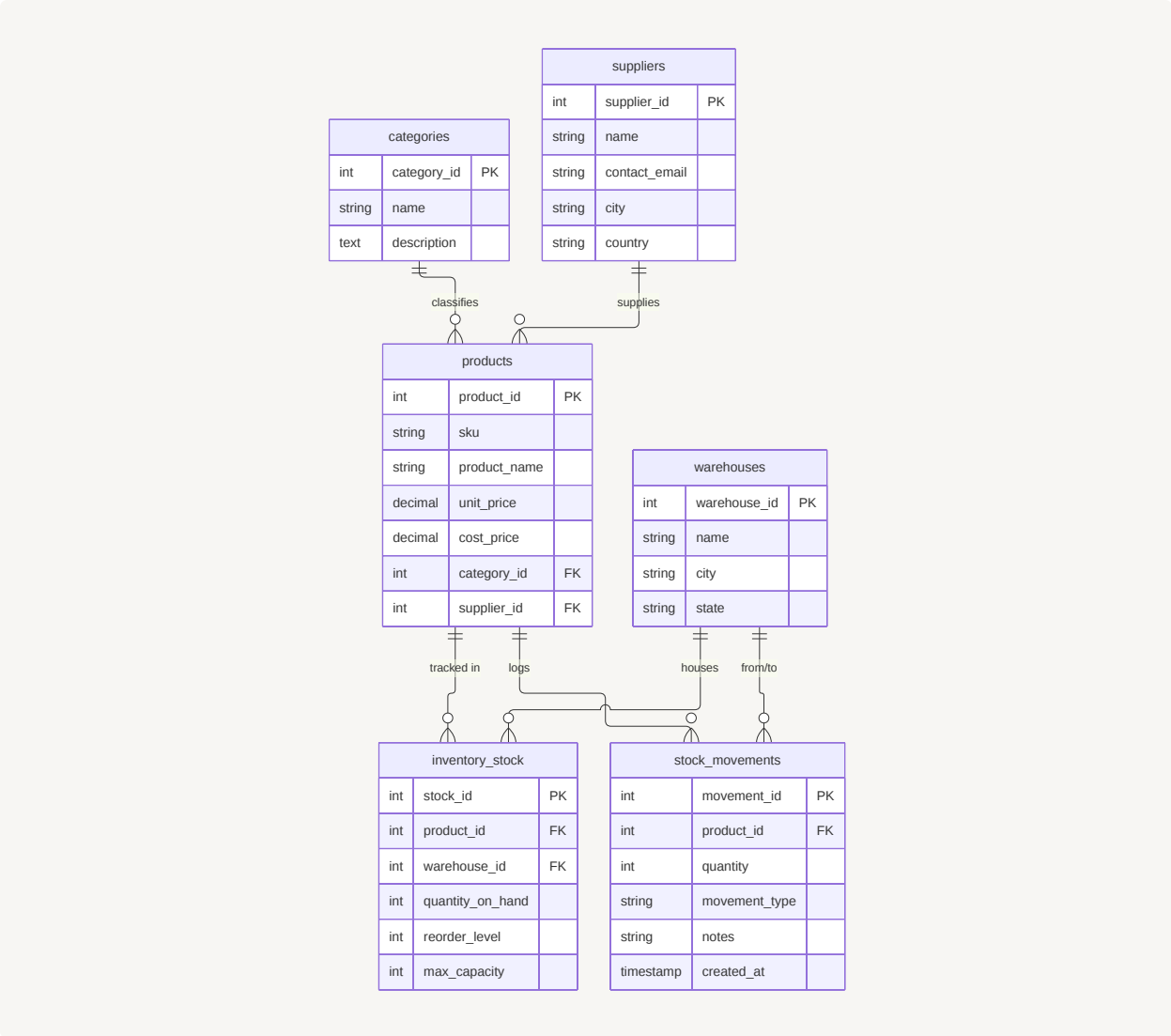
3. [`app.css`](file:///Users/sherlock/Downloads/inventory-agent-app/frontend/src/app/app.css)

- **Glassmorphic UI Design**: Custom color variables (rich dark navy `#0b0f17`, slate `#1e293b`, electric blue `#3b82f6`, and emerald `#10b981`).
- Responsive typography using Google Fonts (Inter and JetBrains Mono).
- Custom scrollbars, status badges, and loading micro-animations.

---

5. Database Schema (`db/schema.sql`)

The database represents a real-world multi-warehouse distribution network:



Summary of Why This Architecture is Robust

1. **Pure Core Java**: Zero Spring Boot overhead. Starts in less than 0.5 seconds, has minimal memory consumption, and uses standard `com.sun.net.httpserver` and JDBC.
2. **Schema Grounding**: Gemini is not hallucinating database records; it is writing real SQL that runs directly against MySQL.
3. **Double Verification**: Business users read the executive analysis, while technical auditors can expand the "Grounding Verification" drawer to inspect the exact SQL query and extracted rows.
4. **Resilient LLM Routing**: If one model hits free-tier quota limits (e.g., 20 requests/day), the client automatically fails over to active models (`gemini-3.5-flash-`

`lite`), preventing downtime.